

Contents

SOQL for Admins: sf CLI Workflow	1
Prerequisites	1
What the challenge wants	1
1. Create the Apex class file	2
2. Create the Apex metadata file	2
3. Deploy the class to your org	3
4. Run the method with anonymous Apex	3
5. Optional: create sample Account data if needed	3
6. Useful verification commands	4
7. Fast Neovim workflow	4
8. Minimal command list	4
9. Expected local files	5

SOQL for Admins: sf CLI Workflow

This README translates the Trailhead unit **Create SOQL Queries in Apex Classes** into a Neovim + Salesforce CLI workflow. The module is part of **SOQL for Admins**, and the unit asks you to create an Apex class named **AccountUtility** with a public static void method named **viewAnnualRevenue** that queries **Account** records for **Name** and **AnnualRevenue**, loops through the results, stores **Account Name : Annual Revenue** in a variable named **acctRev**, and prints it with **System.debug**.

The Salesforce CLI supports both creating local Apex files and running Apex code, including anonymous Apex and test commands, which makes it a good replacement for the Developer Console workflow described in Trailhead.

Prerequisites

Make sure Salesforce CLI is installed, your org is authenticated, and you are inside a Salesforce DX project directory.

Recommended setup commands:

```
sf org list
sf config set target-org=trailheadPlayground
mkdir -p force-app/main/default/classes
mkdir -p scripts/apex
```

`sf org list` shows your authorized orgs, and `sf config set target-org=...` lets later commands use your default org automatically.[4]

What the challenge wants

The unit's class challenge requires:

- Class name: `AccountUtility`.
- Method name: `viewAnnualRevenue`.
- Method modifiers: `public static void`.
- A list named `accountsList` containing a SOQL query for `Name` and `AnnualRevenue` from `Account`.
- A loop over the query results.[2]
- A string variable named `acctRev` that stores `Account Name : Annual Revenue`.
- A debug statement that prints `acctRev`.[2]

1. Create the Apex class file

Open the class in Neovim:

```
nvim force-app/main/default/classes/AccountUtility.cls
```

Paste this Apex code:

```
public with sharing class AccountUtility {
    public static void viewAnnualRevenue() {
        List<Account> accountsList = [
            SELECT Name, AnnualRevenue
            FROM Account
        ];

        for (Account acct : accountsList) {
            String acctRev = acct.Name + ' : ' + acct.AnnualRevenue;
            System.debug(acctRev);
        }
    }
}
```

This implementation matches the challenge requirements for the class name, method signature, queried fields, loop, `acctRev` variable, and debug output format.[2][6]

2. Create the Apex metadata file

Salesforce source format stores Apex classes as a `.cls` file plus a matching `-meta.xml` file.[7][4]

Create the metadata file:

```
nvim force-app/main/default/classes/AccountUtility.cls-meta.xml
```

Paste this content:

```
<?xml version="1.0" encoding="UTF-8"?>
<ApexClass xmlns="http://soap.sforce.com/2006/04/metadata">
    <apiVersion>61.0</apiVersion>
```

```
<status>Active</status>
</ApexClass>
```

3. Deploy the class to your org

Deploy the file with Salesforce CLI:

```
sf project deploy start --source-dir force-app/main/default/classes/AccountUtility.cls
```

The `project deploy start` command deploys local project metadata to your org.[4]

If you prefer deploying the whole classes folder instead, use:

```
sf project deploy start --source-dir force-app/main/default/classes
```

4. Run the method with anonymous Apex

Trailhead usually has you call the class method through an anonymous Apex block when you want to see debug output from a class method.[3]

Create a small runner script:

```
nvim scripts/apex/run_account_utility.apex
```

Paste:

```
AccountUtility.viewAnnualRevenue();
```

Run it:

```
sf apex run --file scripts/apex/run_account_utility.apex
```

The `sf apex run --file` command executes anonymous Apex from a local file, which is the CLI equivalent of opening Execute Anonymous in the Developer Console and clicking Execute.

5. Optional: create sample Account data if needed

If your org has no useful account data yet, you can create a couple of records with Salesforce CLI so the debug output is easier to verify. Salesforce CLI supports data manipulation commands alongside development workflows.

Example:

```
sf data create record --subject Account --values "Name='Acme Tea' AnnualRevenue=500000"
sf data create record --subject Account --values "Name='Herbal House' AnnualRevenue=1250000"
```

Then run the anonymous Apex file again:

```
sf apex run --file scripts/apex/run_account_utility.apex
```

You should see `USER_DEBUG` lines similar to:

Acme Tea : 500000
Herbal House : 1250000

6. Useful verification commands

To confirm the class is present locally:

```
ls force-app/main/default/classes/AccountUtility.cls  
ls force-app/main/default/classes/AccountUtility.cls-meta.xml
```

To stream or inspect logs while you test:

```
sf apex tail log --color  
sf apex log list
```

Salesforce CLI includes Apex log commands for viewing and streaming debug logs in the terminal.[3]

To query Accounts directly from the CLI:

```
sf data query --query "SELECT Name, AnnualRevenue FROM Account" --result-format table
```

Salesforce CLI supports SOQL-based data queries, which is useful for checking the exact records your Apex method should iterate over.

7. Fast Neovim workflow

A tight Neovim workflow for this unit is:

1. Edit `force-app/main/default/classes/AccountUtility.cls`.
2. Save the file.
3. Run your deploy mapping or execute `sf project deploy start --source-dir force-app/main/default/classes/AccountUtility.cls`.
4. Edit `scripts/apex/run_account_utility.apex` if needed.
5. Run `sf apex run --file scripts/apex/run_account_utility.apex`.
6. Read the `USER_DEBUG` output in the terminal.

If you already added Neovim commands like `:SfDeployFile` and `:SfApexRun`, the loop becomes:

```
:edit force-app/main/default/classes/AccountUtility.cls  
:write  
:SfDeployFile  
:edit scripts/apex/run_account_utility.apex  
:write  
:SfApexRun
```

8. Minimal command list

If you only want the exact commands to complete the module workflow, use this sequence:

```
sf org list
sf config set target-org=trailheadPlayground
mkdir -p force-app/main/default/classes scripts/apex
nvim force-app/main/default/classes/AccountUtility.cls
nvim force-app/main/default/classes/AccountUtility.cls-meta.xml
sf project deploy start --source-dir force-app/main/default/classes/AccountUtility.cls
nvim scripts/apex/run_account_utility.apex
sf apex run --file scripts/apex/run_account_utility.apex
```

9. Expected local files

Your project should end up with at least these files:

```
force-app/main/default/classes/AccountUtility.cls
force-app/main/default/classes/AccountUtility.cls-meta.xml
scripts/apex/run_account_utility.apex
```

That structure matches Salesforce DX source format for an Apex class plus an anonymous Apex runner script.